

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1

Case Study

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Assessment Tool Weightage: 25%

QUESTIONS:

Total Marks: 25

Case Study 1: CNN for Automated Medical Image Classification

A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

CNN for Automated Medical Image Classification

A **Convolutional Neural Network (CNN)** is well suited for automatically classifying chest X-ray images into **Normal** and **Disease-Affected** categories. CNNs can learn useful visual patterns directly from X-ray images without requiring manually designed features.

1. Suitable CNN Architecture

A suitable architecture can be designed as follows:

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected → Dropout → Output

Example architecture:

Layer	Configuration	Purpose
Input	$224 \times 224 \times 1$	Grayscale chest X-ray
Conv1	32 filters, 3×3	Detect basic edges and textures
ReLU	—	Introduce non-linearity
Max Pooling	2×2	Reduce spatial dimensions
Conv2	64 filters, 3×3	Learn more complex patterns
ReLU	—	Non-linear activation
Max Pooling	2×2	Reduce feature-map size
Conv3	128 filters, 3×3	Detect disease-related structures
ReLU	—	Non-linearity
Max Pooling	2×2	Reduce computation
Flatten	—	Convert feature maps into a vector
Dense	128 neurons	Perform high-level classification
Dropout	0.5	Reduce overfitting
Output	1 neuron + Sigmoid	Normal/Disease classification

For multi-class disease classification, the final layer can use **Softmax** instead of Sigmoid.

2. Motivation and Role of CNN Components

Motivation for using CNNs

CNNs are preferred because chest X-rays contain spatial patterns such as:

- Lung boundaries
- Opacities
- Consolidation
- Abnormal textures

- Lesions and other disease-related regions

Instead of manually extracting these features, a CNN automatically learns them from training images.

Convolutional Layers

A convolutional layer applies small filters/kernels to the image.

For example, a **3×3 filter** moves across the image and detects specific patterns.

Early layers learn:

- Edges
- Lines
- Simple textures

Deeper layers learn:

- Shapes
- Lung structures
- Abnormal patterns
- Disease-related features

Filters

Each filter learns a different feature. For example, one filter may learn to detect horizontal edges while another learns a particular texture.

During training, the filter weights are automatically updated to identify features useful for classification.

Pooling Layers

Max pooling, commonly with a 2×2 window, reduces the spatial size of feature maps.

Benefits include:

- Reduces computational cost
- Reduces the number of parameters
- Provides some tolerance to small image shifts
- Helps retain important features

Fully Connected Layers

After convolution and pooling, the feature maps are flattened into a vector.

The fully connected layer combines the learned features and determines whether the image is likely to be **Normal** or **Disease-Affected**.

The final **Sigmoid** neuron produces a probability, for example:

Output = 0.92 → Disease-Affected

3. Parameter Sharing

Parameter sharing is one of the major advantages of CNNs.

In a fully connected network, every neuron is connected to every pixel. Therefore, the number of parameters becomes extremely large.

For example, if an image has:

$$224 \times 224 = 50,176 \text{ pixels}$$

and a fully connected layer contains 1,000 neurons:

$$\text{Parameters} \approx 50,176 \times 1,000 = 50 \text{ million}$$

This requires a large amount of memory and computation.

In a CNN, a 3×3 filter contains only:

$$3 \times 3 = 9 \text{ weights} + 1 \text{ bias}$$

The same filter is reused at different locations of the image. Therefore, instead of learning separate weights for every image location, CNNs learn a small set of filters that are **shared across the entire image**.

Thus, parameter sharing:

- Greatly reduces trainable parameters
- Reduces memory requirements
- Makes training faster
- Helps the network detect the same feature regardless of its location

4. Regularization Techniques

Medical image datasets can contain many images, but the model can still **overfit** by memorizing training images instead of learning general disease patterns.

a) Dropout

Dropout randomly disables a percentage of neurons during training.

For example:

Dropout = 0.5

means approximately 50% of selected neurons are randomly ignored during each training step.

This prevents neurons from becoming overly dependent on each other and improves generalization.

b) Data Augmentation

Data augmentation creates slightly modified versions of existing X-rays.

Examples include:

- Small rotations
- Horizontal shifts
- Scaling
- Cropping
- Brightness adjustments
- Small translations

This is particularly useful because X-ray datasets may contain variations in brightness, size, positioning and image quality.

However, augmentation should be medically realistic. Excessive transformations could create unrealistic anatomy and negatively affect classification.

c) Batch Normalization

Batch normalization normalizes intermediate activations during training.

Advantages include:

- More stable training
- Faster convergence
- Reduced sensitivity to initialization
- Some regularization effect
- Better generalization

A typical block can therefore be:

Convolution → Batch Normalization → ReLU → Pooling

Other useful techniques include **early stopping, L2 regularization and learning-rate scheduling.**

5. AlexNet vs ResNet

Between **AlexNet** and **ResNet**, **ResNet** would generally be more appropriate for this medical image classification application.

Feature	AlexNet	ResNet
Network depth	Relatively shallow	Much deeper
Feature learning	Good	Excellent
Architecture	Traditional CNN	Residual CNN
Vanishing-gradient problem	More significant in deeper networks	Reduced using skip connections
Computational requirement	Lower	Higher
Complex feature extraction	Moderate	Very strong
Classification performance	Good baseline	Generally better for complex image tasks

Why ResNet is preferred

Chest X-rays contain complex visual information. Disease-related abnormalities may be subtle and can require learning features at multiple levels.

ResNet uses **residual/skip connections**, which allow information and gradients to pass more easily through deep networks.

For example:

Input → Several convolutional layers → + Original input → Output

The shortcut connection helps the network learn residual features and makes training deep models more effective.

A model such as **ResNet-18** or **ResNet-50** can therefore learn:

Low-level features → Textures → Anatomical structures → Complex abnormal patterns → Disease classification

Computational consideration

The main disadvantage of ResNet is that it requires more computational resources than AlexNet, especially deeper variants such as ResNet-50 or ResNet-101.

However, **transfer learning** can significantly reduce this problem. A ResNet pretrained on a large image dataset can be fine-tuned using the chest X-ray dataset. This usually provides better feature extraction than training a CNN completely from scratch.

Final Choice

ResNet is the better choice because:

1. It provides **deeper and more powerful feature learning**.
2. Residual connections make deep-network training easier.
3. It can learn subtle and complex disease patterns.
4. It generally provides **better image classification performance** than older architectures such as AlexNet.
5. Transfer learning can reduce training time and computational requirements.

Therefore, a **pretrained ResNet-18/ResNet-50 with appropriate fine-tuning, data augmentation, batch normalization and dropout** would be a strong choice for automated chest X-ray classification.

Conclusion

CNNs are highly suitable for medical image classification because they automatically learn hierarchical spatial features from X-ray images. Convolutional filters detect increasingly complex patterns, pooling reduces image dimensions, and fully connected layers perform the final classification. **Parameter sharing** significantly reduces the number of trainable parameters, while **dropout, data augmentation and batch normalization** help prevent overfitting. Among AlexNet and ResNet, **ResNet is generally preferred** because of its stronger feature-learning capability, deeper architecture, residual connections and superior classification performance, although it requires more computational resources.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional filters progressively learn **low-level and high-level features**.

3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

CNN-Based Autonomous Vehicle Object Recognition

1. CNN Architecture

A suitable CNN architecture is:

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected → Output

The input can be an RGB image captured by the vehicle camera. The CNN extracts features and classifies objects such as **cars, pedestrians, bicycles, traffic signs, and obstacles**.

Example:

Layer	Example Configuration	Function
Input	$224 \times 224 \times 3$ RGB	Takes camera image
Conv1	32 filters, 3×3	Detects edges and basic patterns
ReLU	—	Adds non-linearity
Max Pooling	2×2	Reduces spatial dimensions
Conv2	64 filters, 3×3	Learns textures and shapes
ReLU	—	Non-linear activation
Max Pooling	2×2	Reduces computation
Conv3	128 filters, 3×3	Learns complex object features
ReLU	—	Non-linearity
Max Pooling	2×2	Feature reduction
Fully Connected	256 neurons	Combines learned features
Dropout	0.5	Reduces overfitting
Output	Softmax	Predicts object class

For example, the output could be:

- Car → 0.85
- Pedestrian → 0.05
- Bicycle → 0.04
- Traffic sign → 0.03

- Other obstacle → 0.03

2. Role of CNN Layers

- **Input Layer:** Accepts the camera image, usually resized to a fixed size.
- **Convolutional Layer:** Uses filters to extract important image features.
- **ReLU Activation:** Adds non-linearity and helps the network learn complex patterns.
- **Pooling Layer:** Reduces the size of feature maps and computational cost.
- **Fully Connected Layer:** Combines the extracted features for classification.
- **Output Layer:** Predicts the object category.

CNNs learn features progressively. **Early layers detect edges and textures**, middle layers detect **shapes and parts**, and deeper layers identify **complete objects** such as cars and pedestrians.

3. Parameter Sharing

In CNNs, the same filter is applied to different locations of an image. This is called **parameter sharing**.

It:

- Reduces the number of trainable parameters.
- Reduces memory and computational requirements.
- Allows the same feature to be detected at different positions.
- Improves translation-related feature detection.

For example, a CNN can recognize a car whether it appears on the left, center, or right side of the image.

4. Overfitting and Regularization

Overfitting occurs when the CNN performs well on training images but poorly on new road images.

Suitable techniques include:

- **Data Augmentation:** Rotation, scaling, cropping, brightness and contrast changes.
- **Dropout:** Randomly disables neurons during training.
- **Batch Normalization:** Stabilizes and speeds up training.
- **L2 Regularization:** Prevents very large weights.
- **Early Stopping:** Stops training when validation performance stops improving.

These techniques help the model work reliably under different lighting, weather and road conditions.

5. AlexNet vs ResNet

Feature	AlexNet	ResNet
Architecture	Traditional CNN	Residual CNN
Depth	Relatively shallow	Much deeper
Feature learning	Good	Better
Training	Easier	More complex but easier for deep networks due to skip connections
Vanishing gradient	More likely	Reduced
Computation	Lower	Higher
Accuracy	Good	Generally higher

ResNet uses **skip/residual connections**, which help information and gradients pass through deep layers. This allows it to learn more complex features effectively.

6. Recommended Architecture

ResNet is recommended for the autonomous vehicle system because it provides better feature learning and generally higher accuracy than AlexNet. It can recognize complex objects even when they are partially hidden or viewed from different angles.

However, autonomous vehicles require **real-time processing**, so a lightweight model such as **ResNet-18 or ResNet-34** is preferable to very deep versions.

Therefore, **ResNet-18/34 provides a good balance between accuracy, computational cost, and real-time performance**, making it more suitable than AlexNet for autonomous vehicle object recognition.

Conclusion

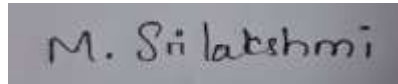
CNNs can automatically learn important visual features and recognize road objects. **Parameter sharing reduces computation**, while regularization prevents overfitting. Among AlexNet and ResNet, **ResNet is the better choice** because of its strong feature-learning ability, reduced gradient problems, and better accuracy with suitable lightweight versions for real-time applications.

Code of Conduct:

I M. Srilakshmi (192525404) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand